

Incident Response Dashboard

Local setup & smoke-test guide — Next.js + Express/Socket.IO + Postgres/Prisma monorepo, mocked GCP/Tableau connectors, real Google OAuth.

1 Install prerequisites

- **Node.js 20 LTS** — nodejs.org. Verify with `node -v` → should print `v20.x.x`.
- **Docker Desktop** (for Postgres) — docker.com. Verify with `docker --version`.

No Docker? Any local Postgres 16 instance works — just point `DATABASE_URL` at it in Step 4 instead of using `docker compose`.

- A **Google account** with access to console.cloud.google.com (for OAuth credentials in Step 3).

2 Install dependencies and start Postgres

From the repo root (`C:\Users\m239012\incident-response-dashboard`):

```
npm install
docker compose up -d postgres
```

Confirm Postgres is healthy: `docker compose ps` should show `postgres` as `healthy` within ~10 seconds.

3 Create a Google OAuth client

1. Go to console.cloud.google.com → select or create a project.
2. **APIs & Services** → **OAuth consent screen** → set it up as **External**; add your own Google account as a test user if it stays in "Testing" mode.
3. **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth client ID**.
4. Application type: **Web application**.
5. **Authorized redirect URIs** — add exactly:

```
http://localhost:3000/api/auth/callback/google
```

6. Save, then copy the generated **Client ID** and **Client Secret** — you'll paste these in Step 4.

4 Create the env files

Because `npm run dev --workspace=X` executes with its working directory set to that workspace, a single root

`.env` is **not** picked up by either app — you need two files, each filled in with real values.

Generate two secrets:

```
openssl rand -base64 32
openssl rand -base64 32
```

Use the first as `NEXTAUTH_SECRET`, the second as `AUTH_JWT_SECRET`. `AUTH_JWT_SECRET` **must be the exact same value in both files below** — it's what lets Express verify tokens issued by Next.js.

`apps/server/.env`

```
DATABASE_URL=postgresql://incident_dash:dev_password@localhost:5432/incident_dashboard

AUTH_JWT_SECRET=<paste secret #2 here>
SEED_ADMIN_EMAIL=<your real Gmail address – the one you'll sign in with>

SERVER_PORT=4000
CORS_ORIGIN=http://localhost:3000

CONNECTOR_MODE_CLOUD_LOGGING=mock
CONNECTOR_MODE_CLOUD_MONITORING=mock
CONNECTOR_MODE_CLOUD_RUN=mock
CONNECTOR_MODE_CLOUD_FUNCTIONS=mock
CONNECTOR_MODE_DATA_FUSION=mock
CONNECTOR_MODE_BIGQUERY=mock
CONNECTOR_MODE_TABLEAU_SERVER=mock

MOCK_GENERATOR_INTERVAL_MS=8000
```

`apps/web/.env.local`

```
GOOGLE_CLIENT_ID=<paste from Step 3>
GOOGLE_CLIENT_SECRET=<paste from Step 3>
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=<paste secret #1 here>

AUTH_JWT_SECRET=<paste secret #2 here – same value as apps/server/.env>

NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_WS_URL=http://localhost:4000
```

(Copying the whole `.env.example` into each file also works fine — unused vars are ignored — but the above is the minimum each app actually reads.)

5 Run migrations and seed the database

```
npm run db:migrate
npm run db:seed
```

`db:migrate` will prompt you to name the initial migration (e.g. `init`) — type a name and press enter. `db:seed` creates starter teams, one runbook per source system, and `ConnectorState` rows defaulted to mock — it does **not** create your admin user yet; that happens automatically on your first real sign-in via `SEED_ADMIN_EMAIL`.

6 Start the app

```
npm run dev
```

This builds `packages/shared` once, then runs the Express/Socket.IO server (port 4000) and Next.js (port 3000) concurrently, labeled `[server]` / `[web]` in the terminal. Wait for `[web]` to print `Ready` and `[server]` to print something like `listening on 4000`.

7 Verify it actually works

Open **two separate browser windows** (or one normal + one incognito) at `http://localhost:3000`:

#	Check
1	Sign in with the Google account matching <code>SEED_ADMIN_EMAIL</code> in window 1 → you should land on the dashboard as Admin.
2	Live feed: within ~8 seconds (the mock generator interval) incidents should start appearing without a refresh, across all 7 source systems.
3	Drill-down: click an incident → confirm the severity/impact factor breakdown renders as a labeled list, and a relevant runbook is suggested.
4	Real-time sync: sign in as the same or another user in window 2, acknowledge an incident in window 1 → window 2's incident list should update live via WebSocket, no refresh.
5	RBAC (UI): in Settings, demote a test user to Viewer, sign in as them in an incognito window → acknowledge/status/settings controls should be hidden.
6	RBAC (server): confirm the server — not just the UI — enforces this (see curl command below). Expect a <code>403</code> .
7	No autonomous remediation: click "Approve remediation" on a suggested step → confirm only a timeline entry + audit log row appears (visible on the History page), and nothing else happens.
8	Reconnect handling: stop the server process → the UI should show a disconnected indicator, then reconnect and refetch cleanly once restarted.

For check #6, grab that Viewer's `session.apiToken` from browser dev tools, then run:

```
curl -X PATCH http://localhost:4000/api/v1/incidents/<id>/status \
  -H "Authorization: Bearer <viewer token>" -H "Content-Type: application/json" \
  -d '{"status":"RESOLVED"}'
```

Troubleshooting

Symptom	Fix
<code>ECONNREFUSED</code> on server start	Postgres isn't up yet; check <code>docker compose ps</code> .
OAuth <code>redirect_uri_mismatch</code>	The redirect URI in Google Cloud Console must be exactly <code>http://localhost:3000/api/auth/callback/google</code> , no trailing slash.
401s from the server / socket won't connect	<code>AUTH_JWT_SECRET</code> differs between the two <code>.env</code> files — they must match exactly.
Migration fails with "database does not exist"	The Postgres container may still be initializing; wait for <code>docker compose ps</code> to show healthy, then retry <code>npm run db:migrate</code> .
Signed in but not Admin	<code>SEED_ADMIN_EMAIL</code> in <code>apps/server/.env</code> must match the exact Google account email you signed in with; promotion only fires on that specific match.